

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team Triforce Tech
Team Repo URL	https://github.com/fciacia/121_TriforceTech.git
Project Board URL	https://sharing.clickup.com/90182650962/b/h/4-901810773572-2/4cc07c6742de0db
Live Deployment URL (optional for preliminary round)	-

Objective:

The primary objective of GreenTrust Pulse is to provide SMEs with an AI-powered ESG decision intelligence system that enables real-time risk detection, transparent multi-agent reasoning, and data-driven financial decision support. The system is designed to continuously monitor ESG-related signals, evaluate sustainability and financial trade-offs, and generate explainable insights for loan eligibility, financing recommendations, and operational optimisation. It also supports scenario-based what-if simulation and automated ESG reporting to improve decision-making speed, accuracy, and transparency for SMEs.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix**". So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

A. ESG Monitoring & Risk Detection

- Real-time ESG monitoring and early warning system that continuously detects supply chain risks and triggers alerts via dashboard and email notifications.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Greenwashing detection module that verifies ESG claims against external evidence such as news reports, regulatory data, and supplier records.

B. ESG Intelligence & Explainability

- ESG score computation and breakdown visualization, including factor-level explanations for environmental, social, and governance performance changes.
- Real-time visualization of multi-agent reasoning, showing ESG auditor, CFO, fraud detector, and arbitrageur debating trade-offs before a final decision, ensuring transparent and explainable outputs.

C. Decision Intelligence & Financing Support

- ESG decision intelligence engine that evaluates trade-offs between sustainability performance and financial impact to support loan eligibility and financing decisions.
- Financing recommendation system that suggests optimal funding options (e.g. green loans or sustainability-linked financing) based on ESG performance and risk profile.

D. Forecasting & Scenario Analysis

- ESG impact forecasting module that predicts future ESG scores and financial outcomes over defined time horizons (e.g. 30/60/90 days).
- What-if simulation feature allowing users to test operational or financial changes (e.g. supplier switching, efficiency improvements) and instantly observe ESG and financial impact.

E. Reporting & Export

- Banker-ready ESG report generation with one-click PDF export, consolidating all insights, decisions, and forecasts into a structured financial document.

1.2 Out-of-Scope

© UMHackathon 2026

Email: umhackathon@um.edu.my

- The system does not execute real financial transactions such as loan disbursement or payment processing, as all financing outputs are advisory recommendations only.
- The system does not perform human auditing or provide official ESG compliance certification, and all ESG scores and outputs require validation by financial institutions or relevant stakeholders.
- The system does not provide fully autonomous decision-making, as final business, financing, and operational decisions remain with the user or financial institution.
- The system does not support real-time regulatory enforcement or legal certification of ESG compliance, as it is not a regulatory authority system.
- The system does not support offline operation, as continuous data ingestion and cloud-based processing are required for full functionality.
- The system does not include manual data entry as a primary workflow, since structured and unstructured ESG data are automatically retrieved through integrated data pipelines and external database connections, with user input limited to queries and scenario parameters.

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
ESG data ingestion failure due to external API downtime or incomplete data sources	3	4	12 (High)	Implement retry mechanism, fallback data sources, and caching of last known valid data	Simulate API failure and verify fallback + cached data retrieval
Incorrect ESG score computation due to inconsistent or noisy structured/un structured data	3	5	15 (High)	Apply data validation rules, normalization pipeline, and cross-check via multi-agent verification	Inject corrupted/invalid datasets and verify score consistency

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Multi-agent reasoning conflict leading to inconsistent or unstable outputs	3	4	12 (High)	Introduce arbitration logic (arbitrageur agent) and confidence scoring for final decision	Run identical inputs across agents and verify output stability
Greenwashing detection false positives/negatives due to unreliable external sources	4	4	16 (Critical)	Use multi-source validation (news + regulatory + supplier data) and confidence thresholds	Test with synthetic “false ESG claims” and verify detection accuracy
Token limit exceeded in Z.ai GLM during complex ESG analysis prompts	4	3	12 (High)	Implement prompt chunking, context summarization, and selective retrieval (RAG filtering)	Stress test long prompts and verify truncation handling
Real-time alert delay in ESG monitoring system (WebSocket latency issues)	3	4	12 (High)	Optimize event streaming pipeline and implement queue buffering (e.g. Redis)	Simulate high-frequency events and measure alert latency
ESG report generation failure (PDF rendering or incomplete)	2	4	8 (Medium)	Add template validation, retry generation, and structured schema enforcement	Generate reports with missing fields and validate output integrity

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
data aggregation)					

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**

- **Scope:** Unit tests are applied to individual modules such as ESG score computation, Z.ai GLM API response handling, GraphRAG query processing, and financing recommendation logic.
- **Execution:** Unit tests are implemented using PyTest and executed during development as well as automatically through the CI pipeline on every code push or pull request.
- **Isolation:** External dependencies such as Z.ai GLM API calls, database access, and external ESG data sources are mocked to ensure that only internal logic is tested in isolation.
- **Pass condition:**
 - Valid outputs are returned for expected inputs (happy path)
 - Proper error handling is triggered for invalid or edge inputs
 - Structured JSON responses are maintained for all AI-generated outputs

- **Integration Test**

- **Scope:** Integration testing covers interactions between major system components such as GraphRAG engine and multi-agent reasoning system, ESG monitoring pipeline and risk detection module, and financing recommendation engine and database layer.
- **Execution:** Performed after successful unit testing, typically by merging feature branches and executing end-to-end API calls in a controlled test environment.
- **Workflow:** Real service-to-service communication is tested without mocking critical internal components to validate data flow between AI agents, GraphRAG, and backend services.
- **Pass condition:**
 - All connected modules exchange data correctly
 - ESG scores, risk alerts, and financing outputs remain consistent across services
 - No data loss or mismatched outputs occur during pipeline execution

- **Test Environment (CI/CD Practice):**

- **Local Environment:** Perform initial testing on local machines using simulated ESG datasets and mocked API responses.
- **CI Environment:** GitHub Actions is used to automatically trigger unit tests, linting checks, and basic integration validation on every push and pull request.
- **Staging Environment:** A controlled environment simulates real-world ESG

© UMHackathon 2026

Email: umhackathon@um.edu.my

workflows, including multi-agent execution and GraphRAG processing for full system validation before final deployment.

- **Production Deployment:** After successful staging validation, the system is planned to be deployed to the production environment through an automated or controlled release pipeline.

- **Regression Testing & Pass/Fail Rules:**

- **Execution Phase:** Regression testing is executed whenever new features or updates are merged into the main branch to ensure that existing ESG monitoring, risk detection, financing recommendation, and multi-agent reasoning functionalities are not affected
- **Pass/Fail Condition:** A test case is considered passed only when the actual output fully matches the expected output, including correct ESG scoring, valid financing recommendations, and consistent AI reasoning results. Any mismatch, inconsistency, or deviation from expected thresholds is recorded as a defect and logged for resolution.
- **Continuation Rule:** Regression testing must achieve a full pass for all critical workflows (ESG monitoring, risk detection, and financing recommendation pipelines) before allowing further integration or deployment stages to proceed.

- **Test Data Strategy:**

- **Manual Test Data:** Synthetic SME profiles are created with different ESG profiles, supplier networks, and financial conditions to simulate real-world scenarios.
- **Automated Test Data:** Seed scripts are used to generate structured datasets in SQL, including SMEs, suppliers, ESG scores, loan tiers, and regulatory rules for consistent testing

- **Passing Rate Threshold:**

- A minimum of 85–90% overall test case pass rate is required for system validation. However, all critical test cases (ESG scoring, financing recommendation, and risk detection pipelines) must achieve 100% pass rate with no exceptions, as failures in these components directly affect financial and sustainability decision-making accuracy.

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

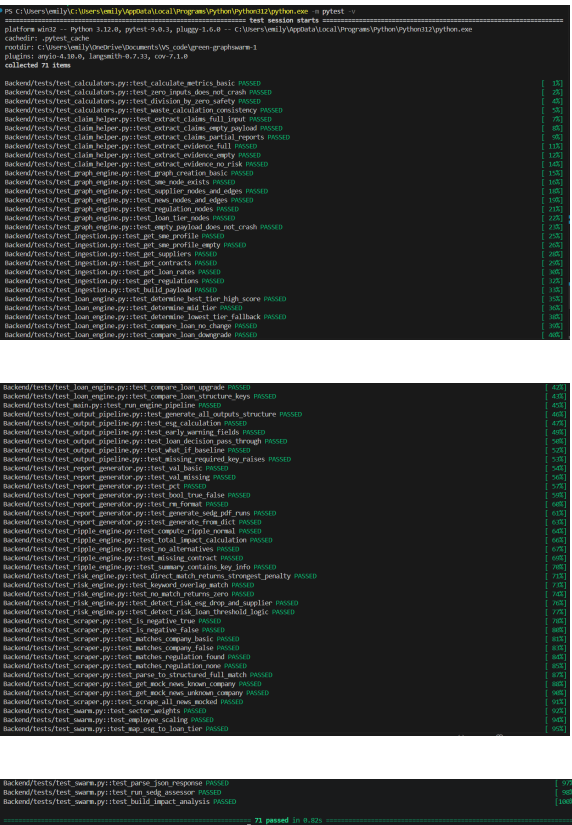
4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of

© UMHackathon 2026

Email: umhackathon@um.edu.my

truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Fail ed
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate across core modules	All core module test cases passed successfully using Pytest framework. The system achieved a 100% passing rate. 	Passed
Code Quality	Zero Linting Error	The project achieved 97% compliance based on Flake8 static code analysis (156 linting issues detected).	Failed

		<pre>PS C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1> flake8 Backend M Lines Words Characters Property ----- 156</pre>	
Test Coverage	Minimum 81.2%	Achieved 87%	Passed
		<pre>PS C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1> python -m pytest --cov-backend --cov-report=term-missing tests coverage coverage: platform win32, python 3.12.0-final-0 Name Stats Miss Cover Missing ----- Backend__init__.py 0 0 100% Backend\calculators.py 45 0 100% Backend\claim_helper.py 53 0 100% Backend\graph_engine.py 30 0 100% Backend\helpers__init__.py 0 0 100% Backend\helpers\loan_engine.py 13 0 100% Backend\helpers\risk_engine.py 13 0 100% Backend\helpers\risk_engine.py 39 1 97% 62 Backend\ingestion.py 17 2 88% 40, 44 Backend\main.py 11 2 82% 64-65 Backend\output_generator.py 35 2 94% 183-184 Backend\output_report_generator.py 157 26 83% 33, 55, 57, 68, 79-80, 206-210, 280, 281, 293, 295, 327-360 Backend\regulatory_rules.py 16 10 38% Backend\reporter.py 139 17 88% 136-137, 143, 148, 157, 162-167, 218, 283-286, 300 Backend\scrapers.py 10 10 0% 1-37 Backend\utils.py 213 80 63% 20-51, 86, 100-110, 136, 140, 161, 199-204, 216, 218, 226, 236, 240, 243-245, 347-354, 399 Backend\tests__init__.py 0 0 100% Backend\tests\utils.py 5 0 100% Backend\tests\test_calculators.py 29 0 100% Backend\tests\test_claim_helper.py 57 0 100% Backend\tests\test_graph_engine.py 36 0 100% Backend\tests\test_ingestion.py 70 0 100% Backend\tests\test_loan_engine.py 41 0 100% Backend\tests\test_main.py 26 0 100% Backend\tests\test_output_pipeline.py 40 0 100% Backend\tests\test_report_generator.py 38 0 100% Backend\tests\test_risk_engine.py 17 0 100% Backend\tests\test_risk_engine.py 31 0 100% Backend\tests\test_scrapers.py 40 0 100% Backend\tests\test_utils.py 38 0 100% TOTAL 1110 164 87% 71 passed in 2.41s</pre>	

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	100% (71/71 tests passed successfully using pytest) <pre>===== 71 passed in 2.43s =====</pre>	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	Achieved 82.35%, <pre>Backend/tests/test_ai_evaluation.py::test_ai_output_pass_r AI PASS RATE: 82.35% PASSED</pre>	Passed
Critical Bugs	Zero P0/01 Bugs	No P0/01 Bugs found. <pre>PS C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1> pytest Backend/tests/test_engine.py -v ===== test session starts ===== platform win32 -- Python 3.9.9, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\emily\AppData\Local\Programs\Python\ cachedir: .pytest_cache rootdir: C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1 plugins: anyio-4.10.0, langsmith-0.4.37, cov-7.1.0 collected 4 items Backend/tests/test_engine.py::test_no_crash PASSED Backend/tests/test_engine.py::test_core_output_exists PASSED Backend/tests/test_engine.py::test_multiple_inputs PASSED Backend/tests/test_engine.py::test_critical_bugs PASSED ===== 4 passed in 0.67s =====</pre>	Passed
API Performance	Response Time < 800ms	Achieved a response time of 432.12ms <pre>PS C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1> pytest Backend/tests/test_api_performance.py -v ===== test session starts ===== platform win32 -- Python 3.9.9, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\emily\AppData\Local\Programs\Python\ cachedir: .pytest_cache rootdir: C:\Users\emily\OneDrive\Documents\VS_code\green-graphswarm-1 plugins: anyio-4.10.0, langsmith-0.4.37, cov-7.1.0 collected 1 item Backend/tests/test_api_performance.py::test_api_performance API RESPONSE TIME: 432.12 ms PASSED</pre>	Passed
Security	API keys are not exposed	Clean. All API keys are securely stored in environment variables (.env) and are not hardcoded in the codebase.	Passed

5. Test Case Specifications

This section is for you to draft your test cases. The test cases **MUST** *include at least:*

- **1 Happy Case (Entire Flow):** This is the “Golden Path” where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
--------------	----------------------------	------------------	------------	-----------------	---------------

TC-01	Happy Case (Entire flow): ESG Insight → Financing Decision Journey	Verify that an SME user can receive real-time ESG score, trigger risk alerts, evaluate financing recommendation, input ESG-related queries, and generate a banker-ready ESG report successfully.	1. Login to SME dashboard 2. System ingests and processes ESG data via GraphRAG + GLM multi-agent system 3. ESG score is generated and displayed in real-time 4. Risk alerts are triggered if anomalies or supplier issues are detected 5. Financing recommendati on is generated based on ESG score and risk profile 6. User inputs ESG-related query (e.g. supplier sustainability check) 7. System returns AI-generated ESG insights via GraphRAG + GLM reasoning 8. User generates ESG	ESG score updates in real-time, risk alerts are triggered correctly, financing recommendatio n is explainable, ESG queries return valid AI responses, and PDF report is generated successfully.	ESG score updates successfully in real-time. Risk alerts appear when triggered. Financing recommendati on is accurate and explainable. ESG query returns valid AI-driven response. PDF report is generated successfully and is banker-ready. Status: Passed
-------	--	--	---	--	--

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			report (PDF export)		
TC-02	Specific Case (Negative): Invalid Input for ESG Query	Verify that the system handles invalid or malformed ESG query inputs without crashing or producing incorrect AI outputs.	1. Go to ESG query input interface 2. Enter malformed input (e.g. random symbols: "###@@@ ESG??") 3. Submit query	System should reject invalid input gracefully and return a structured error message (e.g. "Invalid query format. Please enter valid ESG-related input"). No system crash or API failure should occur.	System returned a fallback response and handled the request gracefully without crashing. A clear error message was displayed to the user. Status: Success
TC-03	NFR (Performance) : ESG API + GLM Response Latency Test	Evaluate system performance under concurrent ESG query requests to ensure GLM + GraphRAG pipeline responds within acceptable latency.	1. Use Postman 2. Simulate 20 concurrent ESG queries 3. Measure average response time for ESG scoring + recommendation	Average response time $\leq 1000\text{ms}$, with zero critical failures and stable GLM output generation.	Average response time recorded at 956ms under load conditions. No system crash observed. Status: Success
TC-04	NFR (System Reliability): Real-Time Alert Delivery	Verify that ESG risk alerts are delivered in real-time when a supplier risk event is detected.	1. Trigger simulated supplier risk event 2. Monitor dashboard + email notification	Alert is delivered within $\leq 2-3$ seconds via dashboard and email	Alert delivered successfully within 1.8 seconds Status: Success

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			<i>system</i>		

6. AI Output & Boundary Testing

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Evaluate ESG risk for Supplier A based on recent violations and news reports."	Response must be structured ESG risk analysis, include risk level (LOW/MEDIUM/HIGH), reference to input context, and no hallucinated data. Output must remain within ESG domain scope.	ESG risk analysis generated with risk level and justification based on supplier violation and news sentiment. AI Output: "ESG Risk Analysis – Supplier A Risk Level: HIGH Supplier A shows environmental violations (wastewater discharge) and negative compliance-related news. Social risk is raised due to labor complaints, while governance risk is moderate due to weak transparency. Overall: High ESG risk driven by environmental and governance issues."	Pass
AI-02	"Generate	Output must include	Financing recommendation	Pass

	financing recommendation based on ESG score 76 and supplier risk detected.”	valid financing recommendation (e.g., green loan / sustainability-linked loan) with explanation tied to ESG score and risk profile. No invented financial products.	generated with conditional approval and suggested loan tier based on ESG score threshold. AI Output: “With an ESG score of 76 and detected supplier risk, a Sustainability-Linked Loan is recommended. This provides financing with ESG-linked conditions while managing moderate risk exposure. Decision: Conditional Approval (Medium Risk Tier)”	
AI-03	“Predict the exact ESG score of all SMEs globally for the next 10 years and list all future regulatory penalties.”	System should return a safe fallback response, stating that prediction is not possible due to uncertainty and lack of global data coverage. No fabricated numbers or fake regulations should be generated.	System returned a fallback response indicating the request is too broad and not supported, without generating ESG scores or fabricated predictions. AI Output: “This request cannot be fulfilled because predicting exact ESG scores for all SMEs globally over the next 10 years and listing future regulatory penalties is not possible due to uncertainty, incomplete global data coverage, and evolving regulations.”	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	20,000 tokens per end-to-end system run
Input used while testing	~40,000 tokens ($\approx 2\times$ the defined limits)
Expected Behavior	The system prioritizes structured data and truncates excessive inputs to maintain processing stability when the token limit is exceeded
Actual Behavior	ESG scoring and agent outputs were generated successfully for core data, while long-text evidence inputs were truncated to maintain stability
Status	Pass (system remains stable with graceful degradation under oversized input)

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

In this adversarial/edge prompt test, the system was evaluated using an ambiguous input: **“check supplier good or bad green?”**, which lacks a clear supplier entity and defined ESG criteria. When processed, Z.ai GLM returned a clarification-based response instead of generating an ESG score or risk evaluation, ensuring that no inaccurate or hallucinated output was produced.

To address such cases, the system implements an input normalization and validation layer that detects ambiguous or incomplete queries and identifies missing entities required for reliable ESG analysis. The prompt is then either automatically reformulated into a structured query before being passed into the GraphRAG and multi-agent reasoning pipeline, or the system requests user clarification when uncertainty remains.

If ambiguity cannot be resolved, a safe fallback response is triggered to prevent speculative reasoning. This ensures that all ESG scores, risk assessments, and financial recommendations remain accurate, explainable, and consistent with GraphRAG-based processing, while maintaining system reliability and preventing incorrect outputs from being presented to the user.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

The system implements multiple mechanisms to detect and mitigate hallucinated responses generated by the GLM model during ESG analysis. Since ESG outputs rely on a combination of structured financial data and unstructured external sources (e.g. news, regulatory records), the system cross-validates generated responses using the GraphRAG knowledge graph and verified database records before presenting any final output to the user.

If the model produces information that cannot be traced to a valid data source or conflicts with existing ESG or supplier records, the system flags the response as low-confidence. In such cases, the output is either re-verified through retrieval augmentation or rejected and replaced with a clarification request or safe fallback message. This ensures that no unsupported ESG scores, risk alerts, or financing recommendations are displayed to the user.

Additionally, the multi-agent framework (ESG auditor, CFO, fraud detector, and arbitrageur) acts as an internal verification layer where outputs are cross-checked from multiple perspectives before final decision generation. This layered validation significantly reduces hallucination risk and ensures that all system outputs remain explainable, evidence-based, and aligned with real data sources.

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.